

SKILL -5

ID-2400031125

NAME-D.M Saifulla

#SKILL - 5 \ Spring Autowiring Demo using @Autowired Prerequisites: • General Idea on Spring Framework • Modules of Spring Framework • Understanding of Autowiring concepts A college application maintains student information along with their certification details. The Student object contains a Certification object, and instead of creating this Certification object manually, Spring should automatically provide it using the @Autowired annotation. When the required classes are marked with @Component, Spring creates their objects and manages them inside the IoC container. Using @Autowired, the Certification object is supplied to the Student object by Spring without using the new keyword, allowing both objects to be linked through annotation-based configuration.

Tasks: 1. Create a Certification class with fields such as id, name, and dateOfCompletion.

2. Create a Student class with fields such as id, name, gender, and a Certification object.

3. Annotate both classes with @Component to enable Spring detection.

4. Use @Autowired on a field, constructor, or setter inside the Student class to inject the Certification object.

5. Configure component scanning using either: a. An XML configuration file b. A Java configuration class with @Configuration and @ComponentScan

6. Load the Spring IoC container using ApplicationContext.

7. Retrieve the Student bean and print all details, including the injected Certification object.

8. Push the Spring project to GitHub.

Step 1: Create the Certification Class

File name: Certification.java

```
package klu.Spring_Boot_Demo;

import org.springframework.stereotype.Component;

@Component
public class Certification {

    private int id;
    private String name;
    private String dateOfCompletion;

    public Certification() {
        this.id = 101;
        this.name = "Java Programming";
        this.dateOfCompletion = "2026-03-01";
    }

    public String toString() {
        return "Certification ID: " + id +
            ", Name: " + name +
            ", Date: " + dateOfCompletion;
    }
}
```

Package: com.example.demo

```
package com.example.demo;
```

```
import org.springframework.stereotype.Component;
```

```
import java.time.LocalDate;
```

```
@Component
```

```
public class Certification {
```

```
    private int id = 101;
```

```
    private String name = "Java Spring Framework";
```

```
    private LocalDate dateOfCompletion = LocalDate.of(2026, 3, 9);
```

```
    public int getId() { return id; }
```

```
    public String getName() { return name; }
```

```
    public LocalDate getDateOfCompletion() { return dateOfCompletion; }
```

```
@Override
```

```

    public String toString() {

        return "Certification [id=" + id + ", name=" + name + ", dateOfCompletion=" +
dateOfCompletion + "]";

    }

}

```

Step 2: Create the Student Class

File name: Student.java

Package: com.example.demo

package com.example.demo;



```

package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class Student {

    private int id = 1;
    private String name = "Rahul";
    private String gender = "Male";

    @Autowired
    private Certification certification;

    public void display() {
        System.out.println("Student ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Gender: " + gender);
        System.out.println(certification);
    }

}

```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Component;
```

```
@Component
```

```
public class Student {
```

```
    private int id = 1;
```

```
    private String name = "Alice";
```

```
    private String gender = "male";
```

@Autowired

private Certification certification; // Spring will inject this automatically

public int getId() { return id; }

public String getName() { return name; }

public String getGender() { return gender; }

public Certification getCertification() { return certification; }

@Override

public String toString() {

return "Student [id=" + id + ", name=" + name + ", gender=" + gender
+ ", certification=" + certification + "];

}

}

Step 3: Create Java Configuration Class

File name: AppConfig.java

Package: com.example.demo

package com.example.demo;

import org.springframework.context.annotation.ComponentScan;

import org.springframework.context.annotation.Configuration;

@Configuration

@ComponentScan(basePackages = "com.example.demo")

public class AppConfig {

```

package klu.Spring_Boot_Demo;

import org.springframework.context.annotation.ComponentScan;

@Configuration
@ComponentScan(basePackages = "klu.Spring_Boot_Demo_")
public class AppConfig {
}
}

```

Step 4: Create Main Class to Run

File name: SpringAutowireDemo.java

Package: com.example.demo

```
package com.example.demo;
```

```
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
public class SpringAutowireDemo {
```

```
    public static void main(String[] args) {
```

```
        // Load Spring IoC container
```

```
        ApplicationContext context = new
        AnnotationConfigApplicationContext(AppConfig.class);
```

```
        // Retrieve Student bean
```

```
        Student student = context.getBean(Student.class);
```

```
        // Print student details including injected Certification
```

```
        System.out.println(student);
```

```

    }
    package klu.Spring_Boot_Demo;
    import org.springframework.context.ApplicationContext;
    import org.springframework.context.annotation.AnnotationConfigApplicationContext;

    public class Main {

        public static void main(String[] args) {

            ApplicationContext context =
                new AnnotationConfigApplicationContext(AppConfig.class);

            Student student = context.getBean(Student.class);
            student.display();
        }
    }

```

```

    }

```

Step 5: Run the Application

Run SpringAutowireDemo.java.

OUTPUT:

Student [id=1, name=Alice, gender=Female, certification=Certification [id=101, name=Java Spring Framework, dateOfCompletion=2026-03-09]]

VIVA VOCE QUESTIONS :

1.What is autowiring in Spring ?

Autowiring is a feature in **Spring Framework** that automatically injects required dependencies into a bean without manually configuring them.

2. What is the purpose of the @Autowired annotation?

Ans: The **@Autowired** annotation tells Spring to automatically inject a required bean into a field, constructor, or method.

3. How does Spring choose which bean to inject?

Ans: Spring selects a bean mainly based on the **dependency type** and sometimes by **bean name** if needed.

4. What is component scanning in Spring?

Ans: Component scanning allows **Spring Framework** to automatically detect classes annotated with @Component, @Service, @Repository, or @Controller.

5. What happens if more than one bean matches the dependency type?

Ans: **If more than one bean matches the dependency type:**

Spring throws an error unless you specify the bean using **@Qualifier** or mark one bean as **@Primary**.

GithubLink: <https://github.com/saifulladm007/FSAD-Skill-Experiments>